Table A.1: CRDTs

CRDT	State	Reducible	Irreducible
G-Counter	Scalar C	*increment*	
PN-Counter	Scalar C	*increment* *decrement*	
LWW-Register	Scalar R Scalar T	*assign*	
G-Set	Set S	*insert*	
PN-Set	Array C		*insert* *remove*
2P-Set	Set A Set R		*insert* *remove*

A Conflict-free Replicated Data Types

A CRDT is a data type (or data structure) that can be replicated across in a distributed computer system in manner that provides several properties. Updates can be performed independently and concurrently at any replica; algorithms within the CRDT that are transparent to the user/programmer propagate updates to all other replicas, resolving any inconsistencies that may occur; and convergence under the strong eventual consistency model is guaranteed. It is important to note that strong eventual consistency *does not* guarantee any timeline by which convergence will be achieved.

Two different CRDT design and implementation strategies have been proposed: operation-[15, 56] and state-based [6, 16]. Both operation-based and state-based CRDTs provide strong eventual consistency, and they are theoretically equivalent, meaning that any operation-based CRDT can emulate its state-based counterpart, and vice-versa [82]. State-based CRDTs require periodic transmission of each replica's state to other replicas, requiring a non-trivial state merge operation, depending on the level of sophistication of the CRDT. In contrast, operation-based CRDTs require each replica to broadcast each method invocation to the other replicas, which increases network communication. The CRDT Wikipedia page[3] provides further details.

A.1 CRDT Benchmark Summary

The CRDTs used for experimental evaluation are described as follows. Our evaluation is limited to scalar instances of each CRDT.

G-Counter[4]: A grow-only counter, which can only be incremented.

PN-Counter: A positive-negative counter which supports increment and decrement operations. A PN-Counter comprises two G-Counters: one for increments, and one for decrements.

LWW-Register: A last writer wins register, which supports assignment of values to the register. Unique timestamps are associated with each assignment, which ensures a total order of operations. The register always retains the most recently written value.

G-Set: A grow-only set which supports insertion, but not removal, of set elements.

PN-Set: A counter is associated with each element of a set: insertion increments the counter; removal decrements the counter. An element is present in the set of its counter is positive. A PN-set may represent a non-traditional multi-set, in which a positive counter value represents the number of occurrences of an element in the set; however, when the counter becomes negative, multiple insertion operations are needed to reintroduce the element to the multi-set.

2P-Set: A two-phase set that supports insertion and removal of set elements. Once removed, an element cannot be reinserted into the set. A 2P-Set is typically implemented using two G-Sets.

[3] https://en.wikipedia.org/wiki/Conflict-free_replicated_data_type

[4] We do not include G-Counter as a benchmark; it is a building block for the PN-Counter.

B Well-Coordinted Replicated Data Types

WRDTs generalize CRDTs with the addition of conflicting methods which require strong consistency [40, 41, 63]. Strong consistency necessitates an SMR protocol to synchronize conflicting calls. Like CRDTs, WRDTs provide strong eventual consistency for non-conflicting method categories. WRDTs thereby guarantee convergence for both conflicting and non-conflicting method categories. WRDTs also provide invariants which are satisfied through permissibility checks, thereby guaranteeing integrity.

B.1 WRDT Benchmark Summary

The WRDTs used for experimental evaluation are described as follows.

Bank Account: A distributed bank account that maintains a scalar balance; $deposit()$ and $withdraw()$ methods respectively increase and reduce the balance. Withdrawals that cause overdrafts (funds less than 0) are impermissible.

Courseware: A university registrar that manages courses to be taught ($addCourse()$, $deleteCourse()$), enrolls students in the University ($addStudent()$), and enrolls students in courses ($enroll()$).

Project: Business software that manages projects ($addProject()$, $deleteProject()$), enrolls personnel ($addEmployee()$) and assigns personnel to work on projects ($assign()$).

Movie: A database of movies ($addMovie()$, $deleteMovie()$) and customers ($addCustomer()$, $deleteCustomer()$).

Auction: An e-commerce site that manages users ($registerUser()$), lists items for direct sale ($sellItem()$), allows users to purchase items ($buyItem()$), and facilitates item sales via auction ($openAuction()$, $placeBid()$, $closeAuction()$).

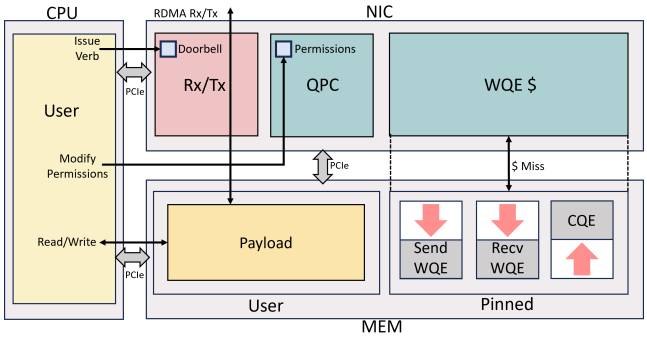
Bank Account's state comprises a single scalar, i.e., one account with one balance. The other WRDTs' states comprise two or three sets, and in the case of Auction, three sets and one array.

C RDMA

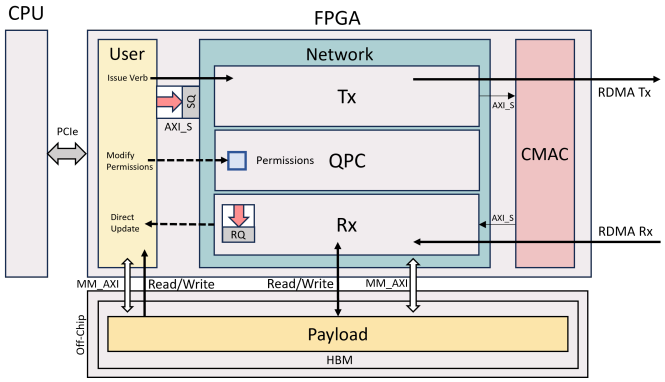
RDMA was originally proposed for high-performance computing (HPC) clusters through the Infiniband protocol, and was later adapted for data center networks as RDMA over Converged Ethernet (RoCE), a link-layer protocol; RoCEv2 is implemented on top of UDP/IPv4 or UDP/IPv6 as an internet-layer protocol, which supports packet routing. Further details can be found on the RoCE Wikipedia page[5].

Table B.1: WRDT Methods and Invariants

RDT	State	Reducible	Irreducible	Conflicting	Sync Group
Bank Account	Scalar B	$deposit(d)$		$withdraw(w)$ where $B - w \geq 0$	1
Courseware	Set S Set C Set E		$addStudent(s)$ where $s \notin S$	$addCourse(c)$ where $c \notin C$ $deleteCourse(c)$ where $c \in C$ $enroll(s, c)$ where $s \in S, c \in C, \langle s, c \rangle \notin E$	1
Project	Set E Set P Set A		$addEmployee(e)$ where $e \notin E$	$addProject(p)$ where $p \notin P$ $deleteProject(p)$ where $p \in P$ $assign(e, p)$ where $e \in E, p \in P, \langle e, p \rangle \notin A$	1
Movie	Set C Set M			$addCustomer(c)$ where $c \notin C$ $deleteCustomer(c)$ where $c \in C$ $addMovie(m)$ where $m \notin M$ $deleteMovie(m)$ where $m \in M$	2
Auction	Set U Set A Set I Array $S[]$	$sellItem(i, u)$	$openAuction(a)$ where $a \notin A$	$registerUser(u)$ where $u \notin U$ $buyItem(i, u)$ where $i \in I, S[i] \geq 1, u \in U$ $placeBid(a, bid, u)$ where $a \in A, u \in U$ $closeAuction(a)$ where $a \in A$	3



(a) Architectural components that implement RDMA for a traditional CPU-based network.



(b) RDMA architecture for a network-attached FPGA featuring a soft RNIC [85].

Figure 18: Architectural components that implement RDMA for both a CPU and FPGA.

C.1 RDMA vs. Socket-based Networking

Figure 18 respectively depicts the components (CPU, memory, NIC) that implement socket-based and RDMA networks; both figures assume that all three components are connected via PCIe.

In Figure 18, the operating system (OS), but not the user-space application, communicates with the NIC. User-space applications must incur the overhead of system calls to transmit and receive data. All data received by the NIC is written to memory managed by the OS kernel; once received, the OS copies data to user-space memory, before the user-space application can access the data.

In Figure 18, the user-space application communicates directly with an RDMA-enabled NIC (RNIC), bypassing the OS kernel. As the RNIC provides remote access to user-space memory, the RNIC's Queue Pair Context (QPC) assumes responsibility for managing memory access permissions, a function typically performed by the OS in other contexts. The three queues that comprise an RDMA queue-pair (QP) for each RDMA connection are allocated in a pinned memory segment. The RNIC contains a cache that provides fast access to QPs. Data is remotely written to user-space memory, eliminating the need to copy data from kernel-managed into user-space memory in Figure 18. User-space applications that expect to receive data poll their memory segments to access data written by remotely-initiated **RDMA_Write**s, and their QPs to detect completion of locally-initiated **RDMA_Read**s.

To refresh terminology, the QP on each side of a connection comprises three queues: a Send Queue (SQ), a Receive Queue (RQ), and a Completion Queue (CQ). The user-space application issues a work request by posting a Work Queue Entry (WQE) to the QP: a Send Queue Entry (SQE) is posted to the SQ, a Receive Queue Entry (RQE) is posted to the RQ, and a Completion Queue Entry (CQE) is posted to the CQ.

C.2 RDMA_READ

Figure 19 depicts the **RDMA_Read** path for a CPU-based RDMA network. The user-application (1) posts a *SQE* and *RQE* to the *SQ* and *RQ* and (2) triggers the RNIC by writing to the doorbell register; both of these steps entail PCIe transactions and the first step accesses memory. The RNIC (3) fetches the *SQE* from memory, (4) checks the QP state, (5) verifies permissions, and (6) packages the **RDMA_Read** operation for transmission over the network; step (4) incurs PCIe transaction and memory access latencies. On the receiving side, the RNIC (7) retrieves and unpacks that **RDMA_Write** operation, (8) verifies permissions, and (9) reads the payload from memory over PCIe at the address specified in the operation The receiver's RNIC (a, b) returns the payload over the network to the sender who (c) writes the received payload to memory, (d) fetches the *RQE* from the *RQ* and (e) writes a *CQE* to the *CQ*, signaling completion of the operation; steps (c), (d), and (e) incur PCIe transaction

[5]https://en.wikipedia.org/wiki/RDMA_over_Converged_Ethernet

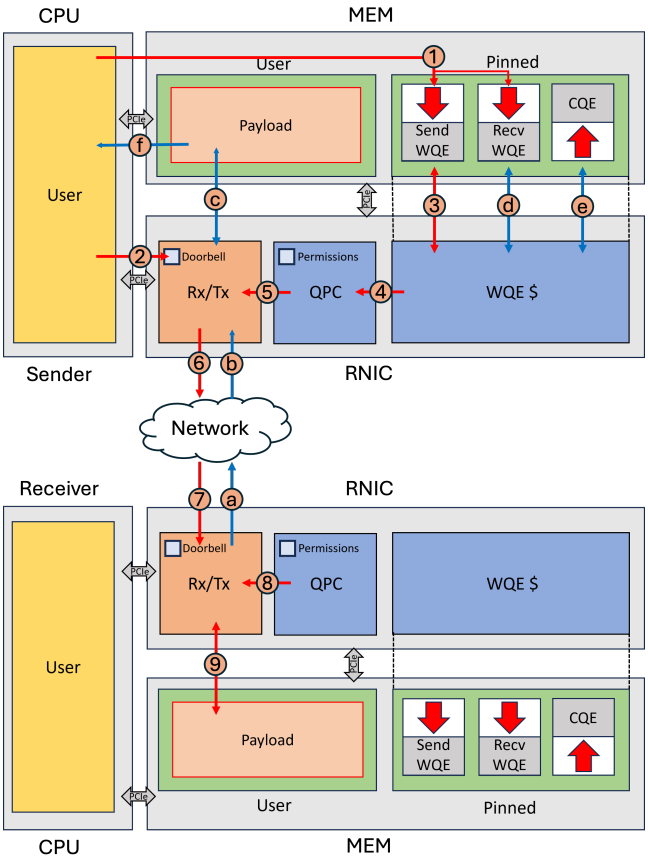


Figure 19: CPU Nic read path

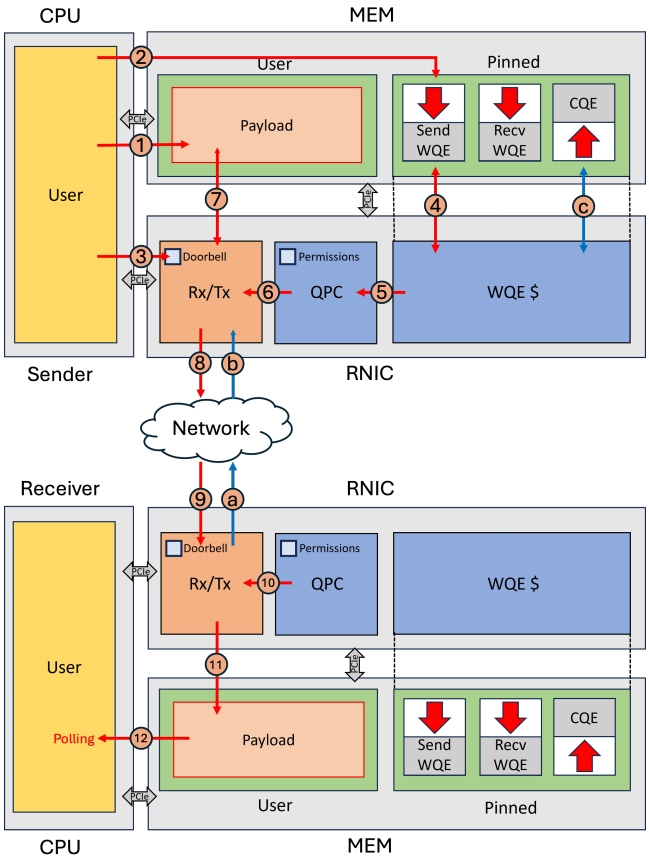


Figure 20: CPU Nic write path

and memory access latencies. The sender's user application can (f) access memory at any time to retrieve the remotely read payload.

C.3 RDMA_WRITE

Figure 20 depicts the **RDMA_Write** path for a CPU-based RDMA network. The user-application (1) writes the payload intended for transmission into memory; (2) posts an *SQE* to the *SQ*; and (3) triggers the RNIC by writing to a doorbell register; all three of these steps entail PCIe transactions, and the first two access memory. The RNIC (4) fetches the *SQE* from memory, (5) checks the QP state, (6) verifies permissions and packet header information, (7) retrieves the payload from memory, and (8) packages the **RDMA_Write** operation for transmission across the network; steps (4) and (7) incur PCIe transactions and memory access latencies. On the receiving side, the RNIC (9) retrieves and unpacks the **RDMA_Write** operation, (10) verifies permissions, and (11) writes the payload to memory over PCIe. The receiver's RNIC (a, b) returns an acknowledgment (ACK) to the sender, whose RNIC completes the **RDMA_Write** operation by posting a CQE to the CQ. The receiver's user application can (12) access memory at any time to retrieve the remotely written payload.

C.4 RDMA_READ (FPGA Implementation)

Figure 21 depicts the **RDMA_Read** path for a pair of network-attached FPGAs.

The user kernel (1) pushes the **RDMA_Read** operation to the *SQ*, implemented as a AXI Stream interface. (2) The Network kernel pops the verb from the *SQ* and (3) checks the *QPC* for permissions, and connection information. Since the operation is **RDMA_Read**, (4) an *RQE* is posted to the *RQ*, which resides in on-chip BRAM. The RDMA packet is (5) packaged and transferred to the CMAC through an AXI Stream. The CMAC takes the RDMA packet, adds Ethernet headers, and (6, 7) transmits the packet over the network. At the receiving node, the CMAC strips the Ethernet header and (8) passes the RDMA packet to the Network kernel. The remote Network kernel (9) checks permissions, and (10) issues the read operation to memory through a memory-mapped AXI interface to obtain the payload, and (a) passes the payload to the network kernel for transmission back to the node that initiated the verb. The RDMA packet is (b, c, d) returned and (e) received by the Network kernel. The Network kernel (f) pops the *RQE* from the *RQ* and writes the payload to the requested address in memory. The User kernel (g) accesses the payload though a memory-mapped AXI interface.

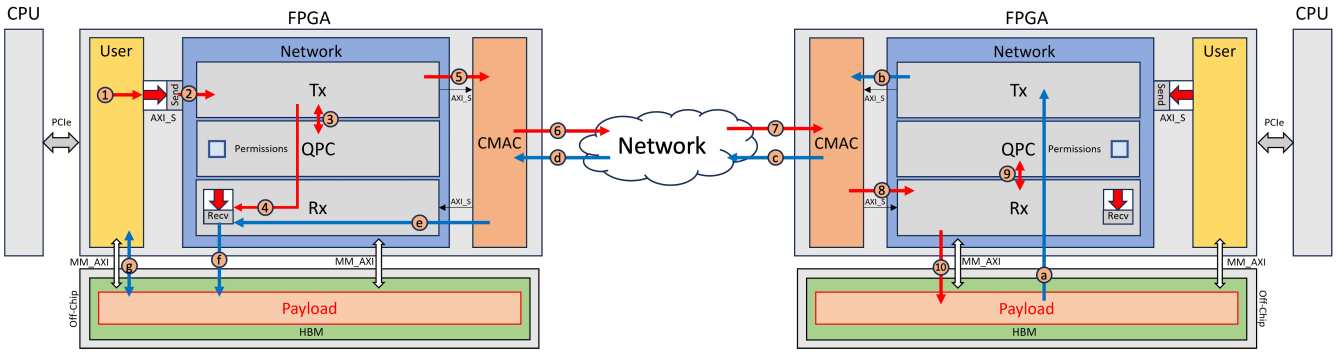


Figure 21: FPGA RDMA read path

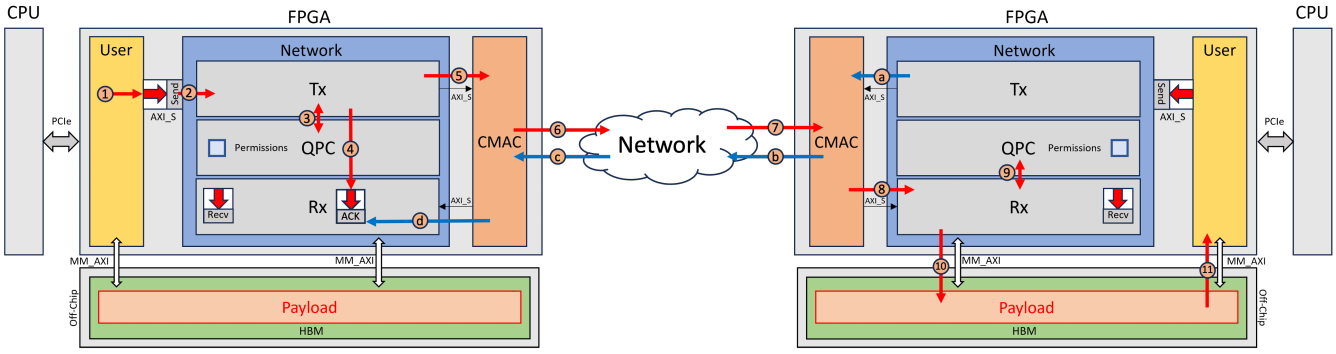
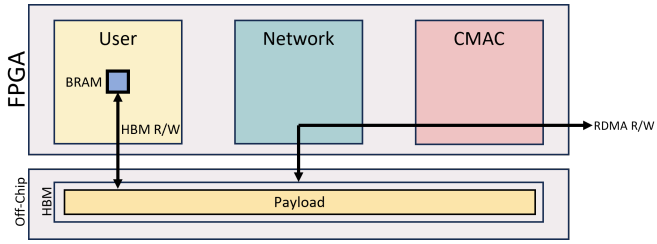


Figure 22: FPGA RDMA write path

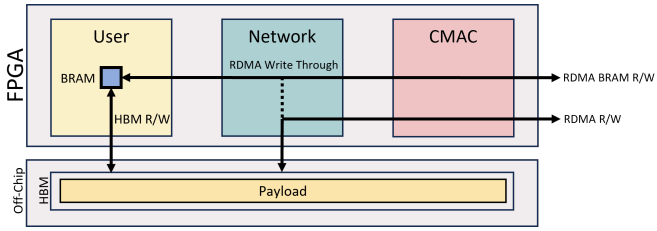
C.5 RDMA_WRITE (FPGA Implementation)

Figure 22 depicts the **RDMA_Write** path for a pair of network-attached FPGAs.

The user kernel (1) pushes the **RDMA_Write** verb to the *SQ*, implemented as a AXI Stream interface. (2) The Network kernel pops the verb from the *SQ* and (3) checks the *QPC* for permissions, and connection information. The Network kernel maintains an queue of expected acknowledgments (ACKs) for retransmission purposes; before transmitting an RDMA packet, the Network kernel posts an expected ACK into the ACK queue. The RDMA packet is (5) packaged and transferred to the CMAC through an AXI Stream. The CMAC takes the RDMA packet, adds Ethernet headers, and (6, 7) transmits the packet over the network. At the receiving node, the CMAC strips the Ethernet header and (8) passes the RDMA packet to the Network kernel. The remote Network kernel (9) checks permissions, and (10) writes the payload to memory through a memory-mapped AXI interface. The User kernel (11) can then access the payload through its own memory-mapped AXI interface. The receiving node generates an ACK and (a) transmits to the CMAC and (b, c) across the network. The sender receives the ACK, notifying it that the **RDMA_Write** verb successfully completed. The Network kernel (d) removes the expected ACK from the ACK queue, thereby completing the operation.



(a) FPGA-based RDMA_Read and RDMA_Write verbs.



(b) FPGA-specific RDMA verbs that access integrated storage.

Figure 23: RDMA verbs for network-attached FPGAs.

Table C.1: Average latencies measured for several RDMA verbs that write to FPGA integrated storage.

Operation	Latency
Write	413 *ns*
BRAM_Write	309 *ns*
BRAM_Write_Through	309 *ns*
Register_Write	285 *ns*
Register_Write_Through	285 *ns*

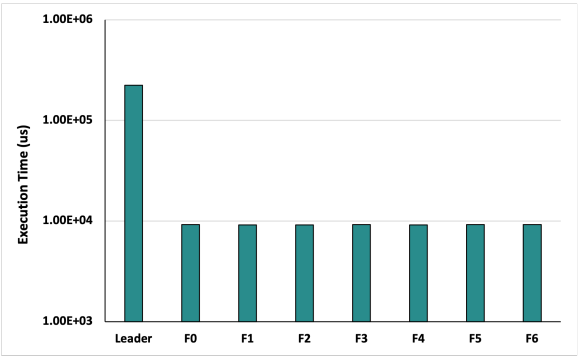


Figure 24: Execution time of all replicas for Bank Account WRDT in a 15% write percentage 8-Node configuration

C.6 FPGA-Specific RDMA Verbs.

Read and Write verbs communicate with remote CPUs via host memory [6]. An analogous implementation of these verbs on a network-attached FPGA utilizes HBM as an intermediary between the local User kernel and the remote node [85] (23).

In 23, we propose RDMA verbs specialized for network-attached FPGAs. Modern FPGAs provide integrated storage. For example, the AMD Ultrascale FPGA in the Alveo U280 accelerator card integrates 2,607,000 registers, 2,016 *Block RAMs (BRAMs)* (36 Kb each) and 960 *UltraRAMs* (288 Kb each); plus any portion of the programmable fabric can be configured as storage. Our proposed FPGA-specific RDMA verbs can read or write to these resources, along with Write-Through verbs which concurrently write to integrated storage and HBM.

Table C.1 reports the latencies of some Write and Write-Through verbs that write to FPGA integrated storage resources. The reported latencies include network transmission, RDMA stack, and HBM, BRAM, or register access latencies. In the case of the Write-Through verbs, the time reported is for BRAM, and a register. Table C.1 *does not* report the time to generate and receive ACKs. We identify one RDT use-case for a write-through verb (subsection 4.3).

D Experiments

D.1 Bank Account: Leaders vs Followers

Figure 24 reports the execution time of the Bank Account WRDT with 8 replicas and a write percentage of 15%. The Leader's execution time is more than twice as larger as the execution times of the seven follower replicas F0-F6; the follower execution times are approximately equal. Throughput is defined as the total execution

[6]RDMA does not support direct access to the CPU register file or cache hierarchy.

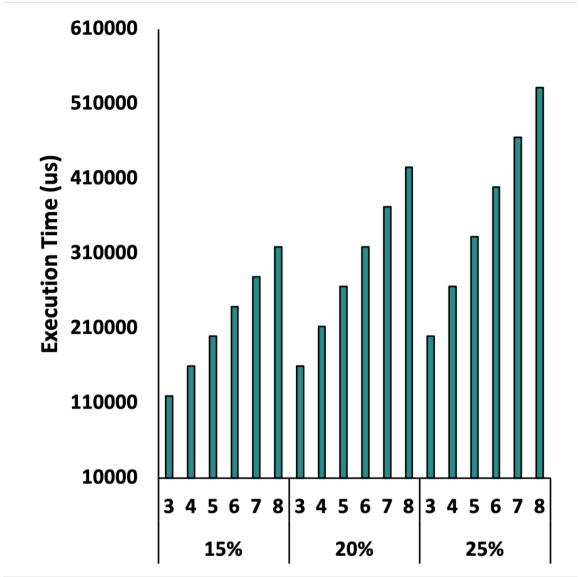


Figure 25: Execution time of leaders in Courseware WRDT

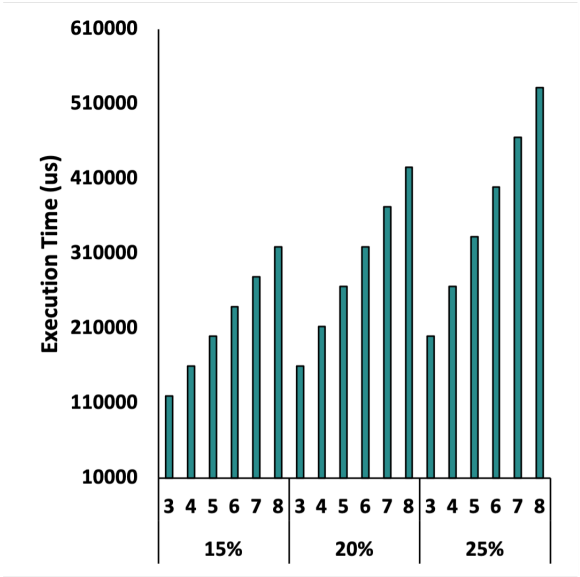


Figure 26: Execution time of followers in Courseware WRDT

time of the workload divided by the total execution time of the system. The total execution time is constrained by the longest-running replica, which is the Leader in Figure 24. Presuming that Bank Account is representative of all WRDTs, Figure 24 indicates that the most promising way to improve throughput is to improve the performance of the Leader, which means focusing on how conflicting transactions and coordination are implemented.

Figure 25 reports the leader execution times across 3-8 replicas and write percentages of 15%, 20%, and 25%, for the Courseware WRDT. Increasing the write percentage tends to increase leader execution time, since there are more conflicting transactions. To

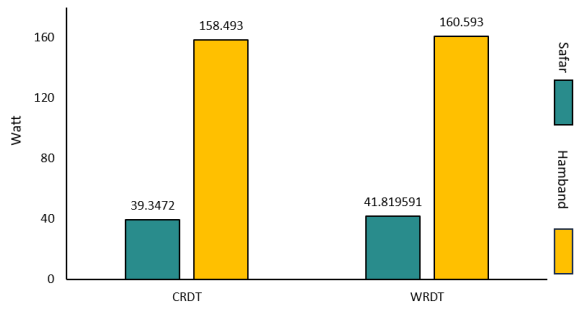


Figure 27: Power Consumption

recall, the write percentage is the fraction of methods that write state, which includes reducible, irreducible conflict-free, and conflicting methods, whereas, all remaining calls invoke the *query*() method. Increasing the number of replicas tend to increase leader execution time, due to the increased cost of coordinating a larger number of followers.

Conversely, Figure 26 reports the average execution time of all followers across the same range of replicas and write percentages. As the number of replicas increase, each follower invokes fewer calls, which reduces execution time. Increasing the write percentage marginally increases execution by adding more conflict-free transactions to execute; as conflict-free methods lack coordination, the slow down is minimal. Conflicting calls do not impact follower execution time to the same extent that they impact leader execution time.

D.2 Power Consumption

FPGA power consumption was measured following AMD Xilinx Documentation [7]. CPU System power consumption was measured using *powerstat*[49].

Figure 27 reports the peak power consumption of CRDTs and WRDTs averaged across use cases and write percentages. SafarDB's power consumption includes the entire Alveo U280 card's power, while Hamband's is the full system including CPU, RNIC, and memory. SafarDB consumes about 1/4th the power of Hamband while providing improved performance; this type of disparity is expected for FPGAs due to a number of reasons, including lower operating frequency and the elimination of inefficiencies such as fetching and decoding machine instructions, data transfer up and down the memory hierarchy on the granularity of cache lines, and the need for every arithmetic operation to access the register file [28, 38].